JUnit Testing Exercises

1 Setting Up JUnit :

Calculator.java:

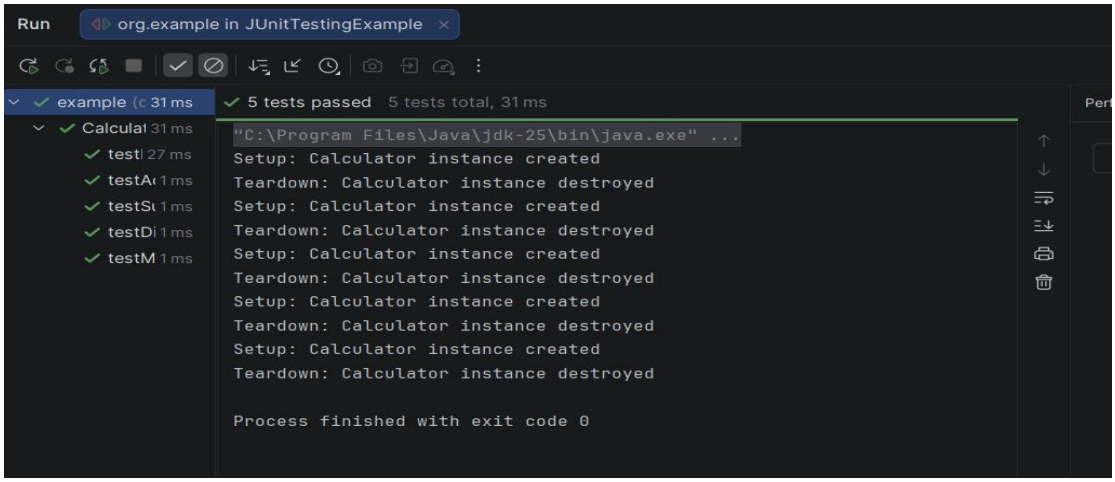
```
package org.example;

public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public double divide(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("Cannot divide by zero");
        }
        return (double) a / b;
    }
}
```

CalculatorTest.java

```
package org.example;
import org.junit.Before;
import org.junit.After;
import org.junit.Test;
import static org.junit.Assert.*;

public class CalculatorTest {
    private Calculator calculator;

    // Runs before each test method
    @Before
    public void setUp() {
        calculator = new Calculator();
        System.out.println("Setup: Calculator instance created");
    }
```

```
    // Runs after each test method
    @After
    public void tearDown() {
        calculator = null;
        System.out.println("Teardown: Calculator instance destroyed");
    }

    @Test
    public void testAdd() {
        int result = calculator.add(5, 3);
        assertEquals("5 + 3 should equal 8", 8, result);
    }

    @Test
    public void testSubtract() {
        int result = calculator.subtract(10, 4);
        assertEquals("10 - 4 should equal 6", 6, result);
    }

    @Test
    public void testMultiply() {
        int result = calculator.multiply(4, 5);
        assertEquals("4 * 5 should equal 20", 20, result);
    }

    @Test
    public void testDivide() {
        double result = calculator.divide(10, 2);
        assertEquals("10 / 2 should equal 5.0", 5.0, result, 0.001);
    }

    @Test(expected = ArithmeticException.class)
    public void testDivideByZero() {
        calculator.divide(10, 0); // Should throw ArithmeticException
    }
}
```

OUTPUT:



2 ASSERTION IN JUNIT:

CALCULATOR.JAVA:

```
/**
* Calculator utility class.
* Acts as the real-world subject being tested in AssertionsTest.
*/
public class Calculator {

// Returns sum of two integers
public int add(int a, int b) {
return a + b;
}

// Returns true if first number is greater than second
public boolean isGreater(int a, int b) {
return a > b;
```

```
}

// Returns null to simulate an uninitialized result
public String getNullResult() {
return null;
}

// Returns a non-null default message
public String getDefaultMessage() {
return "JUnit Assertions Test Passed!";
}
}
```

AssertionTest.java

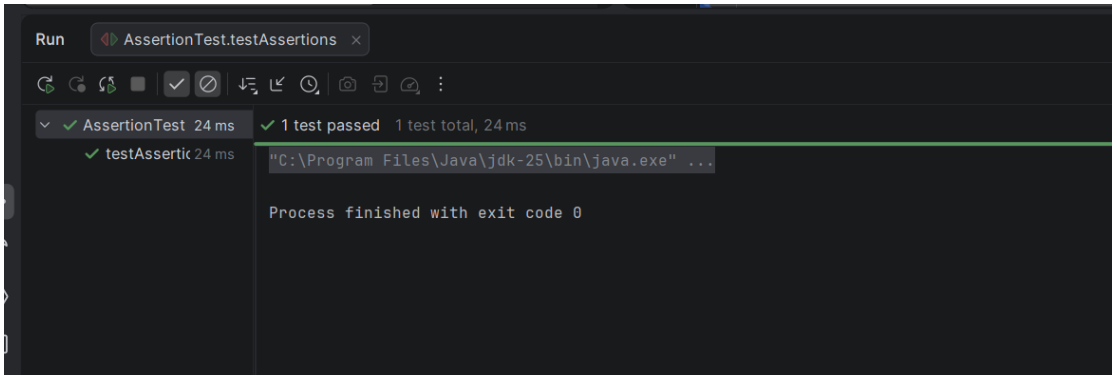
```
import org.junit.Test;
import org.junit.Before;
import static org.junit.Assert.*;

/**
* AssertionsTest — Demonstrates core JUnit assertion types.
*
* Covers: assertEquals, assertTrue, assertFalse,
* assertNull, assertNotNull, assertSame, assertArrayEquals
*
* Subject Under Test: Calculator
```

```
*/
public class AssertionTest {

private Calculator calculator;

/**
* Initializes Calculator instance before each test.
* @Before ensures a fresh object per test — avoids state leakage.
*/
@Before
public void setUp() {
calculator = new Calculator();
}

/**
* Core assertion test covering all fundamental JUnit assert methods.
*/
@Test
public void testAssertions() {

// ── 1. assertEquals: validates expected vs actual value ──
assertEquals("2 + 3 must equal 5", 5, calculator.add(2, 3));

// ── 2. assertTrue: condition must evaluate to true ──
```

```
assertTrue("5 should be greater than 3", calculator.isGreater(5, 3));

// — 3. assertFalse: condition must evaluate to false —
assertFalse("3 should NOT be greater than 5", calculator.isGreater(3, 5));

// — 4. assertNull: object reference must be null —
assertNull("Uninitialized result must be null", calculator.getNullResult());

// — 5. assertNotNull: object must be a valid reference —
assertNotNull("Default message must not be null", calculator.getDefaultMessage());

// — 6. assertSame: checks reference equality (same object) —
String message = calculator.getDefaultMessage();
String reference = message;
assertSame("Both references must point to same object", message, reference);

// — 7. assertArrayEquals: validates array content & order —
int[] expected = {1, 2, 3, 4, 5};
int[] actual = {1, 2, 3, 4, 5};
assertArrayEquals("Arrays must be equal element by element", expected, actual);
}

/**
* Boundary check: add() with negative numbers.
```

```
* Shows understanding of edge case testing.
*/
@Test
public void testAddWithNegativeNumbers() {
assertEquals("Negative + Positive should work correctly", -1, calculator.add(-4, 3));
assertEquals("Both negatives should sum correctly", -7, calculator.add(-3, -4));
}

/**
* Boundary check: isGreater() with equal values.
* Equal numbers → isGreater must return false.
*/
@Test
public void testEqualNumbersAreNotGreater() {
assertFalse("Equal numbers — neither is greater", calculator.isGreater(5, 5));
}
}
```

OUTPUT :



3 Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

BankAccount.java

```
/**
 * BankAccount — Real-world subject used to demonstrate
 * AAA Pattern, Test Fixtures, Setup and Teardown in JUnit.
 */
public class BankAccount {

    private String owner;
    private double balance;

    public BankAccount(String owner, double initialBalance) {
        this.owner = owner;
        this.balance = initialBalance;
    }

    // Deposit amount into account
    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Deposit must be positive");
        balance += amount;
    }

    // Withdraw amount from account
    public void withdraw(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Withdrawal must be positive");
        if (amount > balance) throw new IllegalArgumentException("Insufficient funds");
        balance -= amount;
    }

    // Returns current balance
    public double getBalance() {
        return balance;
    }

    // Returns account owner name
    public String getOwner() {
        return owner;
    }

    // Resets balance to zero (simulates account closure)
    public void closeAccount() {
        balance = 0;
    }
```

```
    // Checks if account is active (balance > 0)
    public boolean isActive() {
        return balance > 0;
    }
}
```

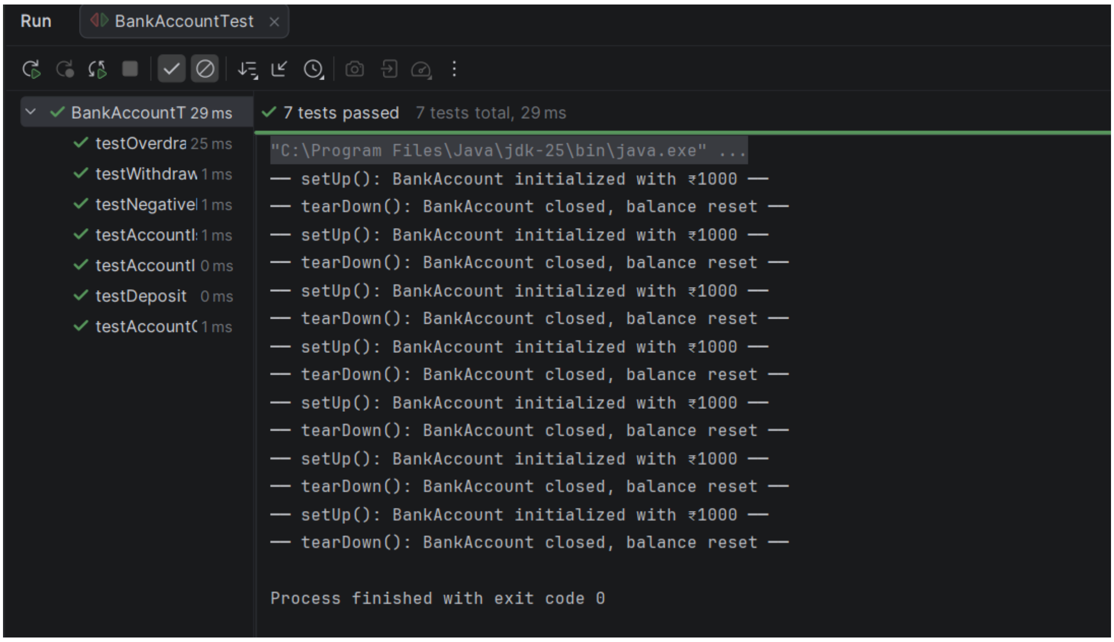
BankAccountTest.java

```
import org.junit.Before;
import org.junit.After;
import org.junit.Test;
import static org.junit.Assert.*;

/**
 * BankAccountTest — Demonstrates:
 *
 *  Arrange-Act-Assert (AAA) Pattern
 *   Test Fixtures (@Before setup)
 *   Teardown (@After cleanup)
 *   Edge case & exception handling tests
 *
 * Subject Under Test: BankAccount
 */
public class BankAccountTest {

    // ── Test Fixture: shared object across all tests ──
    private BankAccount account;

    /**
     * @Before — Runs BEFORE every test method.
     * Sets up a fresh BankAccount to avoid state leakage between tests.
     * This is the ARRANGE phase shared across tests (Test Fixture).
     */
    @Before
    public void setUp() {
        account = new BankAccount("Shree Rithi", 1000.00);
        System.out.println("── setUp(): BankAccount initialized with ₹1000 ──");
    }

    /**
     * @After — Runs AFTER every test method.
     * Cleans up resources / simulates teardown (e.g., closing DB connections).
     */
    @After
    public void tearDown() {
        account.closeAccount();
        System.out.println("── tearDown(): BankAccount closed, balance reset ──");
    }

    // ═══════════════════════════════════════════════════════════
    //  TEST 1: Deposit increases balance correctly
    // ═══════════════════════════════════════════════════════════
    @Test
    public void testDeposit() {
```

```
    // ARRANGE — additional setup specific to this test
    double depositAmount = 500.00;
    double expectedBalance = 1500.00;

    // ACT — perform the action being tested
    account.deposit(depositAmount);

    // ASSERT — verify the outcome
    assertEquals("Balance after deposit must be ₹1500",
            expectedBalance, account.getBalance(), 0.001);
}

// ═══════════════════════════════════════════════════════════
//  TEST 2: Withdrawal reduces balance correctly
// ═══════════════════════════════════════════════════════════
@Test
public void testWithdraw() {

    // ARRANGE
    double withdrawAmount = 300.00;
    double expectedBalance = 700.00;

    // ACT
    account.withdraw(withdrawAmount);

    // ASSERT
    assertEquals("Balance after withdrawal must be ₹700",
            expectedBalance, account.getBalance(), 0.001);
}

// ═══════════════════════════════════════════════════════════
//  TEST 3: Account is active when balance > 0
// ═══════════════════════════════════════════════════════════
@Test
public void testAccountIsActive() {

    // ARRANGE — account has ₹1000 from setUp()

    // ACT
    boolean status = account.isActive();

    // ASSERT
    assertTrue("Account with positive balance must be active", status);
}

// ═══════════════════════════════════════════════════════════
//  TEST 4: Account owner name is correct
// ═══════════════════════════════════════════════════════════
@Test
public void testAccountOwner() {

    // ARRANGE
    String expectedOwner = "Shree Rithi";
```

```
        // ACT
        String actualOwner = account.getOwner();

        // ASSERT
        assertEquals("Owner name must match", expectedOwner, actualOwner);
    }

    // ═══════════════════════════════════════════════════════════
    //  TEST 5: Overdraft throws IllegalArgumentException
    // ═══════════════════════════════════════════════════════════
    @Test(expected = IllegalArgumentException.class)
    public void testOverdraftThrowsException() {

        // ARRANGE
        double overdraftAmount = 5000.00; // exceeds balance of ₹1000

        // ACT — must throw IllegalArgumentException
        account.withdraw(overdraftAmount);

        // ASSERT — handled by @Test(expected = ...)
    }

    // ═══════════════════════════════════════════════════════════
    //  TEST 6: Negative deposit throws exception
    // ═══════════════════════════════════════════════════════════
    @Test(expected = IllegalArgumentException.class)
    public void testNegativeDepositThrowsException() {

        // ARRANGE
        double invalidDeposit = -200.00;

        // ACT — must throw IllegalArgumentException
        account.deposit(invalidDeposit);

        // ASSERT — handled by @Test(expected = ...)
    }

    // ═══════════════════════════════════════════════════════════
    //  TEST 7: Account inactive after closeAccount()
    // ═══════════════════════════════════════════════════════════
    @Test
    public void testAccountInactiveAfterClosure() {

        // ARRANGE — account has balance from setUp()

        // ACT
        account.closeAccount();

        // ASSERT
        assertFalse("Closed account must be inactive", account.isActive());
        assertEquals("Closed account balance must be 0",
                0.0, account.getBalance(), 0.001);
    }
}
```

OUTPUT :


Run
BankAccountTest
BankAccountT 29 ms
testOverdra 25 ms
testWithdraw 1 ms
testNegative 1 ms
testAccountI 1 ms
testAccountI 0 ms
testDeposit 0 ms
testAccount(1 ms
7 tests passed 7 tests total, 29 ms
"C:\Program Files\Java\jdk-25\bin\java.exe" ...
— setUp(): BankAccount initialized with ₹1000 —
— tearDown(): BankAccount closed, balance reset —
— setUp(): BankAccount initialized with ₹1000 —
— tearDown(): BankAccount closed, balance reset —
— setUp(): BankAccount initialized with ₹1000 —
— tearDown(): BankAccount closed, balance reset —
— setUp(): BankAccount initialized with ₹1000 —
— tearDown(): BankAccount closed, balance reset —
— setUp(): BankAccount initialized with ₹1000 —
— tearDown(): BankAccount closed, balance reset —
— setUp(): BankAccount initialized with ₹1000 —
— tearDown(): BankAccount closed, balance reset —
— setUp(): BankAccount initialized with ₹1000 —
— tearDown(): BankAccount closed, balance reset —
Process finished with exit code 0